

Semantic branching pipeline

Diverse agentic code generation via semantic entropy clustering

PHASE 1

INPUTS

SWE-bench Verified issue

Problem statement:

"Bug: overlapping cycles produce wrong output in the permutation constructor..."

Infrastructure

vLLM server

Docker env

DeBERTa NLI

SWE-bench

Input: one GitHub issue + repository snapshot in a Docker container

PHASE 2

SEARCH (read-only exploration)

ReAct agent loop

THOUGHT
"The bug is in the cycle constructor → check parse()"

ACTION
grep -m "def _parse"
sympy/combinatorics/

OBSERVATION
parse() found at line 142
validates input cycles...

repeat for up to 30 steps (read-only: grep, find, cat, head, ...)

NLI relevance scoring

Each observation scored against problem statement:

Step 12: "Found parse() validates cycles"
Step 13: "Listed all test files"

rel = 0.82

rel = 0.18

Saturation detection

3 consecutive low-relevance steps (< 0.5)
detected at step 22 → Transition to Phase 3

Output: search report (relevant code locations + context)

Single trajectory t_0 — explores codebase before any code changes

PHASE 3

STRATEGY PROPOSAL (first branching point)

LLM proposes K = 5 diverse fix strategies

S_0

"Fix validation in parse()..."

S_1

"Add input type guard..."

S_2

"Rewrite the loop logic..."

S_3

"Normalize inputs before..."

S_4

"Refactor cycle constructor..."

Bidirectional NLI entailment clustering

(Farquhar et al. 2024 — compare each pair with DeBERTa-large MNL)

i = 0

No clusters exist → S_0 becomes first cluster

$C_0 = \{ S_0 \star \}$

i = 1

Compare S_1 vs C_0 representative (S_0) — two DeBERTa passes

Forward: $\text{classify}(S_0, S_1)$
Backward: $\text{classify}(S_1, S_0)$

→ $P(\text{ent}) = 0.85$
→ $P(\text{ent}) = 0.78$

0.85 > 0.7 ✓ AND 0.78 > 0.7 ✓ → MATCH

$C_0 = \{ S_0 \star, S_1 \}$

i = 2

Compare S_2 vs C_0 rep (S_0)

Fwd: $P(\text{ent}) = 0.15$ Bwd: $P(\text{ent}) = 0.11$

0.15 < 0.7 ✗ → NO MATCH

→ new cluster

$C_1 = \{ S_2 \star \}$

i = 3, 4

S_3 matches C_1 (both address loop logic); S_4 → new cluster C_2

Result

K = 3 clusters from N = 5 strategies

$C_0 = \{ S_0 \star, S_1 \}$

size = 2

$C_1 = \{ S_2 \star, S_3 \}$

size = 2

$C_2 = \{ S_4 \star \}$

size = 1

$H = -\sum p(c) \cdot \ln p(c) = 1.055 \rightarrow H > \tau \rightarrow \text{BRANCH}$

Fork into K = 3 parallel trajectories (one per cluster, using cluster representative)

PHASE 4

PATCH + VERIFY (per-strategy, lazy sequential)

For each strategy cluster (one Docker container at a time):

Strategy C_0

Strategy C_1

Strategy C_2

sequential

Detailed view: Strategy C_0 trajectory

PATCH sub-phase

(write access enabled: sed -i, patch, cat <<EOF)

Steps 1-4:

Read target files, understand code structure

Step 5:

First write command detected! → triggers SDLG diversification (see below)

sed -i "s/old_check/new_check/" file.py

Steps 6+:

Continue implementing fix (each trajectory runs independently in its own container)

SDLG diversification

(Aichberger et al. 2025 — gradient-based token substitution)

Token attribution via DeBERTa gradients

Identify high-impact tokens in THOUGHT:

"Fix validation in the parse function"

← gradient attribution scores rank these tokens as high-impact

Token substitution + LLM completion

"Fix ordering in the parse..."

→ Alt 1

"Fix constructor in the merge..."

→ Alt 2

"Fix normalization in the repr..."

→ Alt 3

N = 5 alternatives total (greedy + 4 substitutions, split across thought-level and code-level)

Intent extraction + semantic clustering

Extract a one-sentence intent summary per alternative (via LLM)

Cluster via bidirectional NLI entailment (same algorithm as Phase 3)

Entropy → branch decision

$H > \tau \rightarrow$ fork SDLG children (one new trajectory per unique non-greedy cluster)

Primary (greedy)

SDLG child D_1

SDLG child D_2

VERIFY sub-phase

Run pytest to check correctness

→ Submit patch via git diff

→ Destroy container

Each trajectory (primary + SDLG children) verified independently, then next strategy starts

Result: K strategies × (1 + SDLG children) trajectories — each produces an independent patch

Hard cap: B = 30 total trajectories per problem instance

PHASE 5

EVALUATION

Collect patches

All trajectory git diffs gathered as predictions

SWE-bench harness

FAIL_TO_PASS unit tests
PASS_TO_PASS regression tests

Metrics

pass@1 (baseline)
diverse-pass@1 (ours)

diverse-pass@1: instance is resolved if ANY trajectory produces a correct patch

Two diversity axes: strategy-level branching (Phase 3) + implementation-level SDLG (Phase 4)

Semantic entropy as the branching criterion at both levels — branch when the model is uncertain, not when outputs differ superficially